LSM-Tree Key-Value Store in C++ with Live Dashboard

A 4-day project plan: build the storage engine behind RocksDB-style databases from scratch, plus a UI that shows its internals live.

Why this project

Adding a UI makes this a much better portfolio piece, because recruiters can see the engine working instead of reading about it. The idea is a **live dashboard that shows the LSM tree's internals**, not just a key-value form. It teaches the most (skip list, Bloom filter, write-ahead log, SSTables, compaction), maps to real systems (RocksDB, LevelDB, Cassandra), and gives you plenty to say in interviews.

Architecture

- **Engine:** the C++17 LSM core, built as a library.
- **Server:** a thin HTTP layer using `cpp-httplib` (header-only) and `nlohmann/json`. It exposes the engine as a REST API.
- **Frontend:** a single-page dashboard that polls the API every second or so. Use React + Vite if you already know it, or plain HTML/JS with Chart.js for zero build tooling. With plain HTML/JS, the C++ binary can serve the page itself, so the whole project runs with one command.

What the dashboard shows

1. **Key-value console:** put, get, delete, and range scan, with the result and latency in microseconds.
2. **MemTable gauge:** a fill bar showing current size against the flush threshold, so you can watch it approach a flush.
3. **SSTable view:** boxes for each on-disk file with key range, size, and entry count. New files appear after flushes, and merged files replace old ones after compaction.
4. **Bloom filter stats:** counters for lookups, true positives, false positives, and disk reads avoided. This is the "aha" panel.

5. **Event log:** a live feed such as "MemTable flushed → sst_004", "Compaction merged 3 files", "WAL replayed 1,204 entries".
6. **Crash and recover button:** kills and restarts the engine without a clean shutdown, then shows the WAL replay restoring the data. This is your best demo moment.
7. **Benchmark runner:** a button that runs N writes and reads and charts the throughput against a `std::map` baseline.

Key design decision for Day 1

Build observability into the engine from the start. Add a `Stats` struct (counters) and a small ring buffer of events, both thread-safe. Every flush, compaction, and Bloom check pushes to them. The UI then becomes a simple `GET /stats` and `GET /events` call, and you avoid retrofitting instrumentation on Day 3.

API sketch

Endpoint	Purpose
`PUT /kv/:key`, `GET /kv/:key`, `DELETE /kv/:key`	Basic operations
`GET /scan?from=&to=`	Range scan
`GET /stats`	MemTable size, SSTable list, Bloom counters
`GET /events?since=`	Event log feed
`POST /compact`	Trigger compaction manually
`POST /crash`	Simulate an unclean shutdown and recover
`POST /bench`	Run the benchmark

4-day plan

Day 1: in-memory core and observability

- CMake, GoogleTest, folder structure, `.clang-format`
- Skip list (insert, get, delete via tombstones, ordered iteration)
- `MemTable` wrapper with size tracking
- `Stats` and the event log
- Unit tests for everything

Day 2: durability

- Write-ahead log: append every write, replay it on startup
- Flush the MemTable to an on-disk SSTable (sorted key-value file plus a sparse index)
- Bloom filter per SSTable, with a configurable false-positive rate
- Test crash recovery by killing the process mid-write

Day 3: reads, compaction, and the API

- Read path: MemTable, then SSTables newest to oldest, with Bloom filter checks first
- Simple compaction that merges SSTables using a k-way merge with a heap, dropping tombstones and old versions
- Range scan iterator
- REST server with all endpoints (test with `curl`)
- Start the HTML skeleton on the evening of Day 3 while the API is fresh

Day 4: dashboard and polish

- Dashboard, benchmark charts, README with screenshots and a demo GIF, CI badge
- Build the UI in this order: console, MemTable gauge, SSTable view, event log. Add Bloom stats, crash, and benchmark panels last.

Protecting the schedule

Day 4 is tight. To stay on track:

- Do single-level compaction only, with no leveled strategy.
- Cut the benchmark panel first if time runs out.

- Keep the frontend dependency-light so there is no tooling to debug.

GitHub and LinkedIn payoff

- **Demo GIF:** show writes filling the MemTable, a flush creating an SSTable, and a crash followed by recovery. Put it at the top of the README. It will do more than any amount of text.
- **Repo name and description:** something specific like `emberdb: LSM-tree key-value store in C++17`, with topics such as `cpp`, `lsm-tree`, `database`, `data-structures`.
- **Commit as you go.** A history of small commits across 4 days looks much better than one giant upload.
- **Numbers in the README:** ops/sec, test count, Bloom filter false-positive rate.
- **CI:** GitHub Actions that builds and runs tests, with a badge in the README.
- **LinkedIn post:** lead with what you learned, add one benchmark number and the repo link.
- **Resume line:** "Built an LSM-tree key-value store in C++17 with skip-list MemTable, WAL crash recovery, Bloom-filtered SSTables, and compaction; reached X ops/sec in benchmarks, with a live web dashboard visualizing engine internals."

Next steps

Start with Day 1: the folder layout, CMake setup, and the skip list and `Stats` class designs.